

# **SIMATS ENGINEERING**

## **ASSESSMENT TOOL 1**

**COURSE NAME: INTERNET PROGRAMMING**  
**COURSE CODE: CSA4305**

---

### **COURSE OUTCOME COVERED**

**CO 3:** Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

---

### **QUESTIONS:**

**Total Marks: 30**

#### **Question 1**

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.

Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

---

#### **Question 2**

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.

Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

---

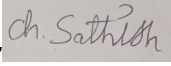
#### **Question 3**

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

**Code of Conduct:**

*I am **Ch.Sathish** / 192311032 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student:** 

**RUBRICS FOR CALCULATION:**

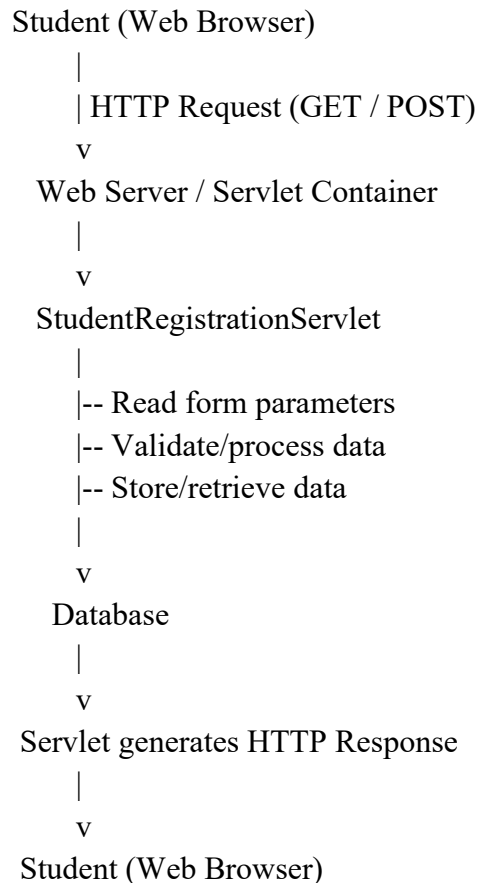
| Criteria                     | Weightage (%) | Level 4 (Exemplary)                                                       | Level 3 (Proficient)                               | Level 2 (Developing)                              | Level 1 (Beginning)                               |
|------------------------------|---------------|---------------------------------------------------------------------------|----------------------------------------------------|---------------------------------------------------|---------------------------------------------------|
| Understanding of Scenario    | 20%           | Demonstrates clear and complete understanding of the scenario and context | Shows good understanding with minor gaps           | Partial understanding; misses key aspects         | Misinterprets or fails to understand the scenario |
| Identification of Key Issues | 20%           | Accurately identifies all relevant issues and factors involved            | Identifies most key issues with minor omissions    | Identifies limited issues; some irrelevant points | Unable to identify key issues                     |
| Application of Concepts      | 25%           | Applies appropriate concepts effectively to address the scenario          | Applies concepts with minor errors                 | Limited or partially correct application          | Unable to apply relevant concepts                 |
| Decision Making              | 20%           | Makes well-reasoned, logical decisions with strong rationale              | Makes reasonable decisions with some justification | Makes weak decisions with limited justification   | No clear decisions made or rationale provided     |
| Clarity of Response          | 15%           | Response is clear, structured, and logically presented                    | Generally clear with minor issues                  | Somewhat unclear or poorly structured             | Disorganized and difficult to understand          |

## ANSWERES

### Question 1: Servlet-Based Application

#### 1. Architecture of a Servlet-Based Application

A servlet-based student registration system generally follows a **client-server architecture**:



#### Main components:

1. **Client/Browser** – Displays the registration form and sends HTTP requests.
2. **Web Server/Servlet Container** – Receives requests and manages the servlet. Examples include Apache Tomcat and Jetty.
3. **Servlet** – Java class that processes the student's submitted information.
4. **Database/Business Layer** – Optionally stores and processes student records.
5. **Response** – The servlet sends an HTML page, JSON, redirect, or other response back to the browser.

## 2.HTML Registration Form

**The same servlet can handle both GET and POST requests.**

```
<!DOCTYPE html>

<html>

<head>

    <title>Student Registration</title>

</head>

<body>

<h2>Student Registration</h2>

<form action="register" method="post">

    <label>Student ID:</label>

    <input type="text" name="studentId"><br><br>

    <label>Name:</label>

    <input type="text" name="name"><br><br>

    <label>Email:</label>

    <input type="email" name="email"><br><br>

    <label>Course:</label>

    <input type="text" name="course"><br><br>

    <input type="submit" value="Register">

</form>

</body>

</html>
```

## 3. Servlet Implementation

Using the Jakarta Servlet API:

```
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
```

```

import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/student")
public class StudentServlet extends HttpServlet {

    @Override
    public void init() throws ServletException {
        System.out.println("StudentServlet initialized");
    }

    @Override
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        String studentId = request.getParameter("studentId");
        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String course = request.getParameter("course");

        out.println("<html><body>");
        out.println("<h2>Student Details - GET</h2>");
        out.println("Student ID: " + studentId + "<br>");
        out.println("Name: " + name + "<br>");
        out.println("Email: " + email + "<br>");
        out.println("Course: " + course + "<br>");
        out.println("</body></html>");
    }

    @Override
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        request.setCharacterEncoding("UTF-8");
        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        String studentId = request.getParameter("studentId");
        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String course = request.getParameter("course");

        // Business logic/database operation can be performed here.

        out.println("<html><body>");
        out.println("<h2>Registration Successful</h2>");
        out.println("Student ID: " + studentId + "<br>");
        out.println("Name: " + name + "<br>");
        out.println("Email: " + email + "<br>");
        out.println("Course: " + course + "<br>");
        out.println("</body></html>");
    }
}

```

```

    }

    @Override
    public void destroy() {
        System.out.println("StudentServlet destroyed");
    }
}

```

## Difference between GET and POST

### GET

- Data is appended to the URL as query parameters.
- Suitable for retrieving data.
- Can be bookmarked and cached.
- Has practical URL-length limitations.
- Should not be used for sensitive information.

Example:

/student?studentId=101&name=John

### POST

- Data is sent in the HTTP request body.
- Suitable for submitting forms and creating/updating data.
- Data is not displayed in the URL.
- Better suited for sensitive form data, although POST alone does not encrypt data.
- HTTPS should be used to protect sensitive information during transmission.

Which method is appropriate for sensitive data?

POST is more appropriate than GET for sensitive form data, because the submitted values are not exposed in the URL, browser history, bookmarks, or many URL-based logs.

However, POST does not provide encryption by itself. For passwords, personal information, or other confidential data, the application should use:

HTTPS + POST

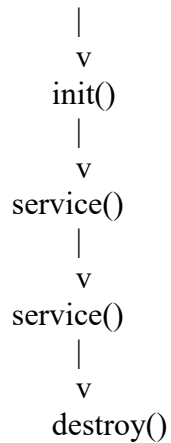
Therefore, sensitive student information should preferably be submitted using **POST over HTTPS**.

## 4. Servlet Life Cycle

The servlet container manages the servlet life cycle.

The main stages are:

Servlet Class Loaded



### 1. init()

The init() method is called **once** when the servlet is initialized.

It is normally used for:

- Initializing resources.
- Reading configuration.
- Creating required objects.
- Establishing resources that can be reused.

```
public void init() throws ServletException {  
    System.out.println("Servlet initialized");  
}
```

### 2. service()

The service() method is called for every request.

For an HTTP servlet, the container's implementation of service() determines whether the request should be processed by:

doGet()

or

doPost()

depending on the HTTP method.

HTTP GET → service() → doGet()

HTTP POST → service() → doPost()

### 3. destroy()

The `destroy()` method is called once before the servlet is removed from service.

It is used for:

- Releasing resources.
- Closing connections.
- Cleaning up objects.

```
public void destroy() {  
    System.out.println("Servlet destroyed");  
}
```

## Conclusion

Servlets provide an efficient way of developing Java web applications. The servlet container manages their life cycle, while methods such as `doGet()` and `doPost()` process client requests. For sensitive information, **POST over HTTPS** is preferred.

## Question 2: Session Management Using HttpSession and Cookies

### 1. Introduction

HTTP is a **stateless protocol**. This means that each request from a browser is treated as an independent request, and the server does not automatically know that two requests belong to the same user.

For example:

```
Request 1 → Login  
Request 2 → View Profile  
Request 3 → View Orders
```

Without session tracking, the server would not automatically know that all three requests belong to the same user.

Therefore, web applications use session tracking mechanisms such as:

- HttpSession
- Cookies
- URL rewriting
- Hidden form fields

For a login system, HttpSession is commonly used, with a session ID usually maintained through a cookie.

### 2. Login Form

```
<!DOCTYPE html>  
<html>
```



```

<head>
  <title>Login</title>
</head>
<body>

<h2>User Login</h2>

<form action="login" method="post">
  Username:
  <input type="text" name="username"><br><br>

  Password:
  <input type="password" name="password"><br><br>

  <input type="submit" value="Login">
</form>

</body>
</html>

```

### 3. Login Servlet Using HttpSession

```

import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

@WebServlet("/login")
public class LoginServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        String username = request.getParameter("username");
        String password = request.getParameter("password");

        // Example authentication
        if ("admin".equals(username) && "1234".equals(password)) {

            HttpSession session = request.getSession();

            session.setAttribute("username", username);
            session.setAttribute("loggedIn", true);

            response.sendRedirect("home");

        } else {
            response.setContentType("text/html");

            PrintWriter out = response.getWriter();
            out.println("<h3>Invalid username or password</h3>");
        }
    }
}

```

In a real application, passwords should not be hard-coded. Authentication should normally be performed against a database using securely hashed passwords.

## 4. Accessing Session Data

A servlet on another page can retrieve the session:

```
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

@WebServlet("/home")
public class HomeServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
        throws ServletException, IOException {

        HttpSession session = request.getSession(false);

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        if (session != null &&
            Boolean.TRUE.equals(session.getAttribute("loggedIn"))) {

            String username =
                (String) session.getAttribute("username");

            out.println("<h2>Welcome, " + username + "</h2>");
            out.println("<p>You are successfully logged in.</p>");

        } else {
            out.println("<h3>Please login first.</h3>");
        }
    }
}
```

## 5. Logout Servlet

A logout servlet can invalidate the session.

```
import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

@WebServlet("/logout")
```

```

public class LogoutServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
        throws ServletException, IOException {

        HttpSession session = request.getSession(false);

        if (session != null) {
            session.invalidate();
        }

        response.sendRedirect("login.html");
    }
}

```

## 6. Session Tracking Using Cookies

A cookie is a small piece of data stored by the browser and sent back to the server with subsequent requests.

Example:

```

import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.Cookie;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/cookie")
public class CookieServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
        throws ServletException, IOException {

        Cookie cookie = new Cookie("username", "John");

        cookie.setMaxAge(60 * 60);
        cookie.setHttpOnly(true);
        cookie.setSecure(true);

        response.addCookie(cookie);

        response.setContentType("text/html");
        response.getWriter().println("Cookie created.");
    }
}

```

A cookie can later be retrieved:

```

Cookie[] cookies = request.getCookies();

if (cookies != null) {

```

```

    for (Cookie cookie : cookies) {
        if ("username".equals(cookie.getName())) {
            String username = cookie.getValue();
            System.out.println(username);
        }
    }
}

```

## 7. How HttpSession Works

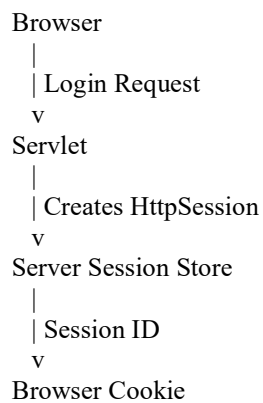
When the application executes:

```
HttpSession session = request.getSession();
```

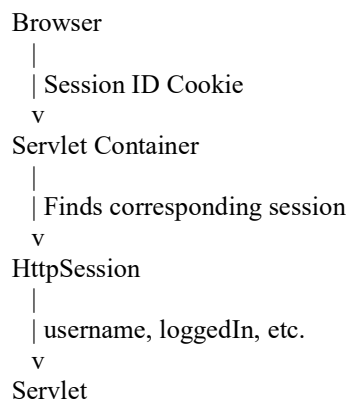
the servlet container creates or retrieves a session.

The container associates the session with a unique session ID.

### Typically:



### For subsequent requests:



Thus, the server can identify the user across multiple requests.

---

## 8. HttpSession vs Cookies

| <b>Feature</b> | <b>HttpSession</b>                          | <b>Cookies</b>                            |
|----------------|---------------------------------------------|-------------------------------------------|
| Storage        | Mainly server-side                          | Client/browser-side                       |
| Identification | Uses a session ID                           | Stores data directly                      |
| Security       | Generally safer for sensitive session data  | Data can be viewed/modified by the client |
| Server memory  | Uses server resources                       | Does not store application data on server |
| Size           | Can hold considerably more server-side data | Limited cookie size                       |
| Lifetime       | Can expire after timeout/invalidation       | Can be session or persistent              |
| Common use     | Login/session state                         | Preferences, session IDs, tracking        |

### **Advantages of HttpSession**

- Convenient for maintaining login state.
- Data is stored on the server.
- Supports multiple attributes.
- Can be invalidated during logout.
- Suitable for shopping carts and authenticated sessions.

### **Advantages of Cookies**

- Simple to implement.
- Data persists between requests.
- Can support persistent information such as preferences.
- Commonly used to carry the session ID.

### **Security considerations**

For authentication systems:

- Use HTTPS.
- Use HttpOnly cookies to reduce JavaScript access to session cookies.
- Use Secure cookies when HTTPS is used.
- Set an appropriate SameSite policy.
- Regenerate the session ID after successful authentication when appropriate.
- Invalidate the session during logout.
- Never store plaintext passwords in cookies.

## Conclusion

For a multi-page login system, HttpSession is the preferred mechanism for maintaining server-side authentication state. Cookies are often used underneath the session mechanism to transport the session ID between the browser and server.

## Question 3: JDBC-Based Product Management Application

### 1. Introduction

**JDBC (Java Database Connectivity)** is a standard Java API used to communicate with relational databases.

An e-commerce application can use JDBC to:

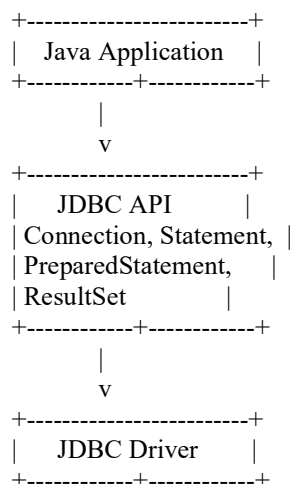
- Connect to a database.
- Insert product information.
- Execute SQL queries.
- Retrieve product details.
- Update or delete products.
- Handle database exceptions.

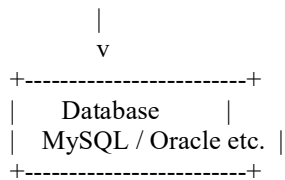
Example product information:

Product ID  
Product Name  
Price  
Quantity

### 2. JDBC Architecture

The JDBC architecture can be represented as:





## Main JDBC components

### 1. DriverManager

DriverManager manages JDBC drivers and establishes database connections.

```
Connection con =
    DriverManager.getConnection(url, username, password);
```

### 2. Connection

Represents a connection between the Java application and database.

```
Connection con;
```

### 3. Statement

Used to execute SQL statements.

```
Statement stmt = con.createStatement();
```

### 4. PreparedStatement

Used for parameterized SQL queries.

```
PreparedStatement ps =
    con.prepareStatement(
        "INSERT INTO product VALUES (?, ?, ?, ?)"
    );
```

PreparedStatement is preferred for user-supplied data because it helps prevent SQL injection.

### 5. ResultSet

Stores the result returned by a SELECT query.

```
ResultSet rs = stmt.executeQuery();
```

### 3. Database Table

For example, a MySQL database can contain the following table:

```
CREATE DATABASE ecommerce;
```

```
USE ecommerce;

CREATE TABLE product (
    id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    price DECIMAL(10,2),
    quantity INT
);
```

#### 4. Steps in JDBC Database Connectivity

The general JDBC process is:

1. Import JDBC packages  
↓
2. Load/register JDBC driver  
↓
3. Establish connection  
↓
4. Create PreparedStatement/Statement  
↓
5. Execute SQL query  
↓
6. Process ResultSet  
↓
7. Close resources

#### 5. JDBC Product Application

The following example inserts and retrieves product information.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class ProductJDBC {

    private static final String URL =
        "jdbc:mysql://localhost:3306/ecommerce";

    private static final String USER = "root";
    private static final String PASSWORD = "password";

    public static void main(String[] args) {

        String insertSQL =
            "INSERT INTO product (id, name, price, quantity) " +
            "VALUES (?, ?, ?, ?)";

        String selectSQL =
            "SELECT id, name, price, quantity FROM product";
```



```

try (Connection con =
    DriverManager.getConnection(URL, USER, PASSWORD);
    PreparedStatement insert =
        con.prepareStatement(insertSQL);
    PreparedStatement select =
        con.prepareStatement(selectSQL)) {

    // Insert product
    insert.setInt(1, 101);
    insert.setString(2, "Laptop");
    insert.setDouble(3, 55000.00);
    insert.setInt(4, 10);

    int rows = insert.executeUpdate();

    System.out.println(
        rows + " product inserted successfully."
    );

    // Retrieve products
    try (ResultSet rs = select.executeQuery()) {

        System.out.println("\nProduct Details:");

        while (rs.next()) {

            int id = rs.getInt("id");
            String name = rs.getString("name");
            double price = rs.getDouble("price");
            int quantity = rs.getInt("quantity");

            System.out.println(
                id + " | " +
                name + " | " +
                price + " | " +
                quantity
            );
        }
    }

} catch (SQLException e) {

    System.err.println(
        "Database error: " + e.getMessage()
    );

    e.printStackTrace();
}
}

```

## 6. Explanation of the Code

### Step 1: Database URL

```

private static final String URL =
    "jdbc:mysql://localhost:3306/ecommerce";

```

This identifies the MySQL database server and database name.

### Step 2: Establish connection

```
Connection con =  
    DriverManager.getConnection(URL, USER, PASSWORD);
```

This establishes a connection between Java and MySQL.

### Step 3: Prepare INSERT query

```
String insertSQL =  
    "INSERT INTO product (id, name, price, quantity) " +  
    "VALUES (?, ?, ?, ?)";
```

The ? symbols represent parameters.

### Step 4: Set parameters

```
insert.setInt(1, 101);  
insert.setString(2, "Laptop");  
insert.setDouble(3, 55000.00);  
insert.setInt(4, 10);
```

The parameters are safely supplied to the prepared statement.

### Step 5: Execute INSERT

```
int rows = insert.executeUpdate();
```

executeUpdate() is used for SQL statements such as:

- INSERT
- UPDATE
- DELETE

### Step 6: Execute SELECT

```
ResultSet rs = select.executeQuery();
```

executeQuery() is normally used for SELECT statements.

### Step 7: Read ResultSet

```
while (rs.next()) {  
    int id = rs.getInt("id");  
    String name = rs.getString("name");  
}
```

ResultSet represents the rows returned by the SELECT query.

### Step 8: Close resources

The example uses **try-with-resources**:

```
try (Connection con = ...;
    PreparedStatement ps = ...) {
```

Resources are automatically closed when execution leaves the block.

## 7. JDBC Exception Handling

Database operations can fail for many reasons:

- Invalid database URL.
- Database server unavailable.
- Incorrect username/password.
- SQL syntax error.
- Duplicate primary key.
- Constraint violation.
- Network failure.

These problems can be handled using `SQLException`.

```
try {
    // JDBC operations
}
catch (SQLException e) {
    System.out.println("Database error: " + e.getMessage());
}
```

Useful methods include:

```
e.getMessage();
e.getSQLState();
e.getErrorCode();
```

For example:

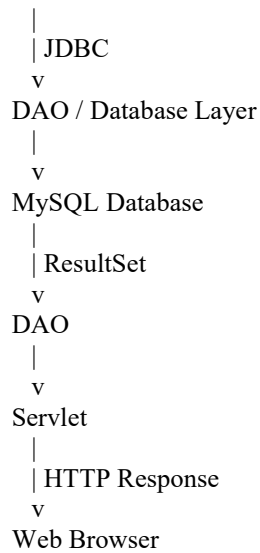
```
catch (SQLException e) {

    System.err.println("Message: " + e.getMessage());
    System.err.println("SQL State: " + e.getSQLState());
    System.err.println("Error Code: " + e.getErrorCode());
}
```

## 8. JDBC Application in a Web Environment

For an e-commerce web application, the architecture can be:

```
Customer
|
v
Web Browser
|
| HTTP Request
v
Product Servlet
```



A typical application separates responsibilities:

### **Servlet**

Handles HTTP requests and responses.

```
@WebServlet("/products")
public class ProductServlet extends HttpServlet {
    // Handle web requests
}
```

### **DAO (Data Access Object)**

Contains database operations.

```
public class ProductDAO {

    public void insertProduct(Product product) {
        // JDBC INSERT operation
    }

    public List<Product> getProducts() {
        // JDBC SELECT operation
    }
}
```

### **Model**

Represents product information.

```
public class Product {

    private int id;
    private String name;
    private double price;
    private int quantity;
```

```
// constructors, getters and setters  
}
```

This separation makes the application easier to maintain and test.

## **9. Important JDBC Best Practices**

1. Use PreparedStatement instead of constructing SQL with string concatenation.
2. Always close Connection, PreparedStatement, and ResultSet.
3. Prefer try-with-resources.
4. Do not hard-code database passwords in production applications.
5. Validate user input.
6. Use transactions when multiple database operations must succeed or fail together.
7. Use a connection pool in production web applications rather than opening a new physical database connection for every request.

## **Conclusion**

JDBC provides a standard mechanism for Java applications to communicate with relational databases. The main steps are establishing a connection, creating a statement, executing SQL, processing the result, and closing resources. In an e-commerce application, JDBC can be combined with Servlets and DAO classes to create a structured system for inserting, retrieving, updating, and deleting product information.